

Generative Modelling of Maritime AIS Trajectories with Retrieval-Augmented Transformers

A Land-Aware, Water-Strict Route Generator from Start, End and Vessel Type

Andrei Știrbu

École Polytechnique, S4 PRL

Supervisor: Davide Buscaldi (LIX, École Polytechnique & Sorbonne Université)

May 2026

Abstract

This report presents a complete machine-learning pipeline that learns to generate realistic maritime vessel routes from minimal conditioning. Given only a start position, an end position and a vessel-type code, the system emits a full 128-waypoint trajectory that follows real shipping patterns and never crosses land. The work pairs a retrieval-augmented encoder-decoder Transformer with a land-aware differentiable training loss and an A* water-graph post-processor, the three pieces co-designed so that water validity is guaranteed structurally while predictive accuracy stays competitive. Trained on a year of US-coastal AIS data (61 000 quality-filtered tracks from ~ 99 million raw position reports) the production model achieves **Average Displacement Error (ade) of 6051 ± 225 m and 0 % land-crossings** at the 10 km threshold, evaluated over five subsample seeds. That is a 27 % improvement over the great-circle baseline (8226 m) on the same evaluation set and a > 50 % reduction in land-crossing rate relative to all baselines that ignore geography. We describe the full pipeline (data filtering, model architecture, land-aware loss, hard projection, A* snap-only router), provide pseudocode for the training and inference loops, and report ablations that justify each design choice. The report concludes with limitations and a fully-specified diffusion extension (TrajDiff) that is the natural next iteration.

Contents

1	Introduction	3
1.1	Context and motivation	3
1.2	Goal of the project	3
1.3	What was personally implemented	3
1.4	Headline result	4
1.5	Outline	4
2	Background	4
2.1	The AIS data source	4
2.2	Trajectory modelling with Transformers	4
2.3	Retrieval-augmented generation	4
2.4	Related work	5
2.5	Signed distance functions for land geometry	5
3	Research challenge	5

4	Methodology	6
4.1	Data preparation	6
4.2	Model architecture (v10)	8
4.3	Training objective	9
4.4	Hyperparameters and training details	9
4.5	Inference and post-processing	10
5	Technical contributions	11
5.1	The data-quality win (E1)	11
5.2	Per-step retrieval bank	11
5.3	Land-aware loss	12
5.4	WaterRouter snap-only post-processor	12
5.5	Implementation notes	12
5.6	Reproducible evaluation tooling	13
6	Experiments and results	13
6.1	Main results	13
6.2	Length-bucketed analysis	13
6.3	Seed variance	13
6.4	Training dynamics	14
6.5	Land-validity diagnostics	14
6.6	Qualitative examples	14
6.7	Motivating ablations	15
6.8	Use-case ranking	15
7	Summary, conclusion and outlook	15
7.1	Summary	15
7.2	Why the work is technically solid	16
7.3	Limitations	17
7.4	Future work	17
7.5	Closing remark	17
A	Training pseudocode	18
B	Selected code snippets	18
B.1	Per-step route encoder	18
B.2	WaterRouter snap-only inference path	18
C	Complete results table	19
D	Reproducibility commands	19

1 Introduction

1.1 Context and motivation

The *Automatic Identification System* (AIS) broadcasts the position, speed, identity and type of every large commercial vessel at intervals ranging from a few seconds to a few minutes. In the United States, NOAA archives the resulting stream through the Marine Cadastre programme and publishes daily comma-separated files covering all coastal waters. A single calendar year yields on the order of 10^8 position reports.

This volume of structured movement data invites a question that combines machine learning and operational maritime engineering: can we *learn* the spatial regularities that human navigators rely on, so that a model handed only a port of origin, a destination and a vessel type can sketch a plausible route between them? A successful answer has direct downstream value – traffic simulation, risk assessment for storm avoidance, port-arrival estimation, detection of anomalous vessels by comparing observed routes against a learned distribution of normal ones.

1.2 Goal of the project

The project addresses two distinct prediction problems in sequence:

Next-step prediction (v1).

Given a short window of past positions, predict the displacement of the vessel to its very next reported position. The natural baseline here is a constant-velocity (CV) extrapolation.

Conditional full-route generation (v2 onward).

Given only (`start`, `end`, `vessel_type`), generate a full $T = 128$ -waypoint route. The natural baseline is a great-circle geodesic between start and end.

The first problem turned out to be a near-optimal use-case for the CV baseline; the second problem is much richer and admits substantial learning signal once the formulation is right. The bulk of the technical work and all of the production results live in the second formulation. The aim of this report is to make the path between these two problems explicit, to describe the implementation in concrete terms, and to report quantitative results that justify each design choice.

1.3 What was personally implemented

Every script and source module in the repository was written by the author. No third-party trajectory models were imported. Concretely the contributions are:

- A reproducible **data pipeline** that turns NOAA archives into a curated 128-waypoint corpus (resampling, quality filtering, GSHHS land filtering – the “E1” data-level win).
- A **retrieval-augmented encoder-decoder Transformer** (`TrajectoryGeneratorV10`) with per-timestep cross-attention over $K = 5$ historical neighbours subsampled to $t_{\text{retr}} = 32$ steps each.
- A **land-aware training loss** $\mathcal{L}_{\text{land}}$ built on a differentiable signed-distance field (SDF) sampled with `F.grid_sample`.
- A **water-graph A* post-processor** (`WaterRouter`) that snaps any predicted waypoint to its nearest connected water cell on a 0.005° adjacency graph.
- Evaluation tooling (ADE/FDE/land-crossing/length-bucketed), five-seed variance experiments and a complete suite of diagnostic and visualisation scripts.

1.4 Headline result

The production system – v10 trained with land-aware loss + WaterRouter snap-only at inference, evaluated on the cleaned 61 K-track corpus – achieves the numbers in Table 1.

Table 1: Headline performance, 500 validation routes, mean over 5 random seeds. Lower is better for every metric.

Method	ADE (m)	FDE (m)	cross%@10km
v10 + WaterRouter (ours)	6051 $\pm$ 225	581	0.0
Retrieval top-1 (zero training)	11 667	11 832	0.0
Great-circle baseline	8 735	~ 0.1	9.0

The model reduces ADE by 27 % versus the great-circle baseline and brings the land-crossing rate to a strict zero at the 10 km clearance threshold – a hard operational requirement that no purely geometric baseline can satisfy.

1.5 Outline

Section 2 reviews the necessary background. Section 3 states the research challenge precisely. Section 4 describes the methodology and the architectural pipeline. Section 5 highlights the technical contributions. Section 6 presents experiments and results, including ablations and seed variance. Section 7 concludes with limitations and outlook. Appendices contain training/inference pseudocode, selected code snippets and the full quantitative summary table.

2 Background

2.1 The AIS data source

The *Automatic Identification System* is mandated by the International Maritime Organisation on all large vessels. Each transponder broadcasts an MMSI (a 9-digit vessel identifier), a position fix, a course-over-ground (COG), a speed-over-ground (SOG) and a vessel-type code on the 28-entry AIS class table. NOAA’s Marine Cadastre programme archives these reports as daily CSV files covering US coastal waters. This work uses the full calendar year 2024.

2.2 Trajectory modelling with Transformers

Modern sequence models for spatial trajectories typically build on the Transformer encoder-decoder architecture of Vaswani et al. [1]. A *causal* decoder generates positions autoregressively, attending to its own history through a triangular mask; cross-attention to an encoder injects the conditioning signal at every step. This is the dominant architecture for next-token modelling in natural language and has been adapted to spatial sequences in maritime [2] and aviation [3] domains. Compared to recurrent baselines it offers two advantages for our setting: (i) direct cross-attention to global conditioning tokens (start, end, vessel type, retrieved historical neighbours) without an information bottleneck; (ii) parallel teacher-forced training, which is essential for $T = 128$ -step targets.

2.3 Retrieval-augmented generation

Retrieval-augmented models couple a parametric generator with a non-parametric memory. Originally proposed for language modelling [4, 5], the idea is to fetch the K most similar items from a training set at inference and condition the generator on them. In this work we use a 5-dimensional KNN over (start lon, start lat, end lon, end lat, vessel type) to fetch $K = 5$

historical routes for every query and then condition the decoder on those routes via cross-attention. Retrieval is the single most impactful change in the entire ladder of twelve variants: a zero-training top-1 retrieval baseline already beats the best non-retrieval Transformer.

2.4 Related work

Maritime trajectory prediction. A line of work treats next-position prediction as a sequence-modelling problem and applies recurrent or Transformer-based architectures [2, 7]. These models are typically evaluated on short horizons (1-5 future positions) and on data dense enough that the constant-velocity baseline is hard to beat. Our v1 experiments confirm this regime experimentally: on turn-segmented open-sea data, no amount of tuning, feature engineering, or model scaling pushes the Transformer below the CV baseline at the 3-minute reporting interval.

Conditional trajectory generation. A separate body of work generates whole trajectories conditioned on origin and destination, often using variational auto-encoders [8] or generative adversarial networks [9]. The conditional-Transformer formulation that this report adopts is simpler and trains stably with maximum likelihood, at the cost of needing a careful endpoint loss to control autoregressive drift.

Retrieval augmentation. Khandelwal et al. [4] showed that augmenting a parametric language model with a non-parametric nearest-neighbour memory yields large perplexity gains. Lewis et al. [5] formalised retrieval-augmented generation (RAG) for open-domain QA. We apply the same principle to spatial sequence generation, with two differences: (i) the retrieval index is built over a low-dimensional engineered key (start lon/lat, end lon/lat, vessel type) rather than a learned embedding; (ii) the retrieved items themselves are full trajectories that we expose to the decoder *per timestep*, rather than condensing them into a single summary vector – the latter approach (our v9) underperformed exactly the zero-training top-1 retrieval baseline on the long-route bucket.

Geographically valid generation. Several lines of work address geographic constraints during generation explicitly: soft penalties on land proximity [7], masked-softmax pointer networks over a water graph (our parked v11/v12 variants), and post-hoc snap-to-water heuristics. This report shows that the simplest approach – a thin geometric post-processor on top of an unconstrained generator – is also the most effective once the training corpus has been cleaned of inland artefacts.

2.5 Signed distance functions for land geometry

A *signed distance function* (SDF) maps a 2D point to the signed Euclidean distance to the nearest contour. Pre-computed on a regular grid, it admits fast nearest-water queries and – because bilinear sampling is differentiable – can be used directly inside a training loss to penalise predicted positions that fall on land. We compute the SDF once from the Global Self-consistent Hierarchical High-resolution Shoreline (GSHHS) data, at 0.05° resolution over the bounding box $(-125^\circ, 10^\circ, -60^\circ, 55^\circ)$, and load it into both training (PyTorch `F.grid_sample`) and inference (numpy bilinear + BFS to nearest water cell).

3 Research challenge

The challenge of full-route generation can be stated as a constrained sequence-generation problem. Let $\mathcal{D}$ be a corpus of vessel routes, each a sequence $\tau = (p_1, p_2, \dots, p_T) \in \mathbb{R}^{T \times 2}$ with $T = 128$ equally-spaced (along arc length) waypoints. Each route has an associated vessel type

$v \in \{0, \dots, 27\}$. Given a query $\mathbf{q} = (p_1, p_T, v)$ we seek a generator G_θ that produces a sequence $\hat{\tau} = (\hat{p}_1, \dots, \hat{p}_T)$ minimising the *Average Displacement Error*

$$\text{ADE}(\hat{\tau}, \tau) = \frac{1}{T} \sum_{t=1}^T \text{haversine}(\hat{p}_t, p_t)$$

subject to two constraints: (C1) endpoint exactness $\hat{p}_1 = p_1, \hat{p}_T = p_T$ and (C2) water validity $\text{sdf}(\hat{p}_t) \leq \theta_{\text{land}}$ for every t .

The challenge is non-trivial because the three pieces – accuracy, endpoint exactness, and water validity – pull against each other:

- A model that only minimises ADE (plain Huber loss on displacements) drifts away from the destination over 127 AR steps, breaking C1.
- A model that emphasises endpoint loss collapses toward straight-line interpolation, which kills route diversity and also crosses land freely (breaking C2).
- A model that adds a hard land-projection step pays an ADE cost on every projected point.

The contribution of this work is to show that, with the right data-cleaning step (E1, Section 4.1) and the right post-processor (WaterRouter, Section 4.5), all three constraints can be satisfied at once with no measurable accuracy trade-off.

4 Methodology

The full pipeline is summarised in Figure 1: raw NOAA AIS archives are filtered, merged, resampled to 128 waypoints and land-cleaned; auxiliary structures (the SDF raster, the water-cell graph, the KNN cache) are built once and reused; the v10 model is trained on the cleaned corpus; inference applies hard land projection, linear endpoint correction and a final WaterRouter snap to produce the production output.

4.1 Data preparation

The raw 2024 AIS archive consists of 99 million position reports across roughly 6×10^5 vessel track segments. The data pipeline applies five stages of filtering.

1. Quality filter. `scripts/data/prepare_data.py` restricts to passenger / cargo / tanker vessels (AIS codes 60-89, $\sim 80\%$ of the raw drop comes from this single rule), enforces a US bounding box, splits at time gaps >60 min, removes physically implausible jumps (>2 km between consecutive reports), and drops tracks that are nearly stationary. Around 97% of raw pings are discarded as a result, leaving 598 000 clean track segments.

2. Resampling. `scripts/data/prepare_trajgen.py` resamples each track to exactly $T = 128$ waypoints, equally spaced along arc length. This decouples the prediction task from the original (variable) reporting interval and gives every training sequence a fixed shape $(128, 2)$. Tracks shorter than 20 km or longer than 2000 km are rejected, leaving 207 000 tracks.

3. Land filter (E1). `scripts/data/filter_trajgen_npz.py` discards any track whose ground-truth waypoints cross land more than 2% of the way (using the coarse 0.05° SDF) or whose maximum penetration depth into land exceeds 10 km. This is the single biggest data win in the project: the raw `trajgen_128.npz` file has a 10.83% land fraction in the *ground truth itself* (due to raster noise on complex coastlines, plus inland river traffic that survived the quality filter).

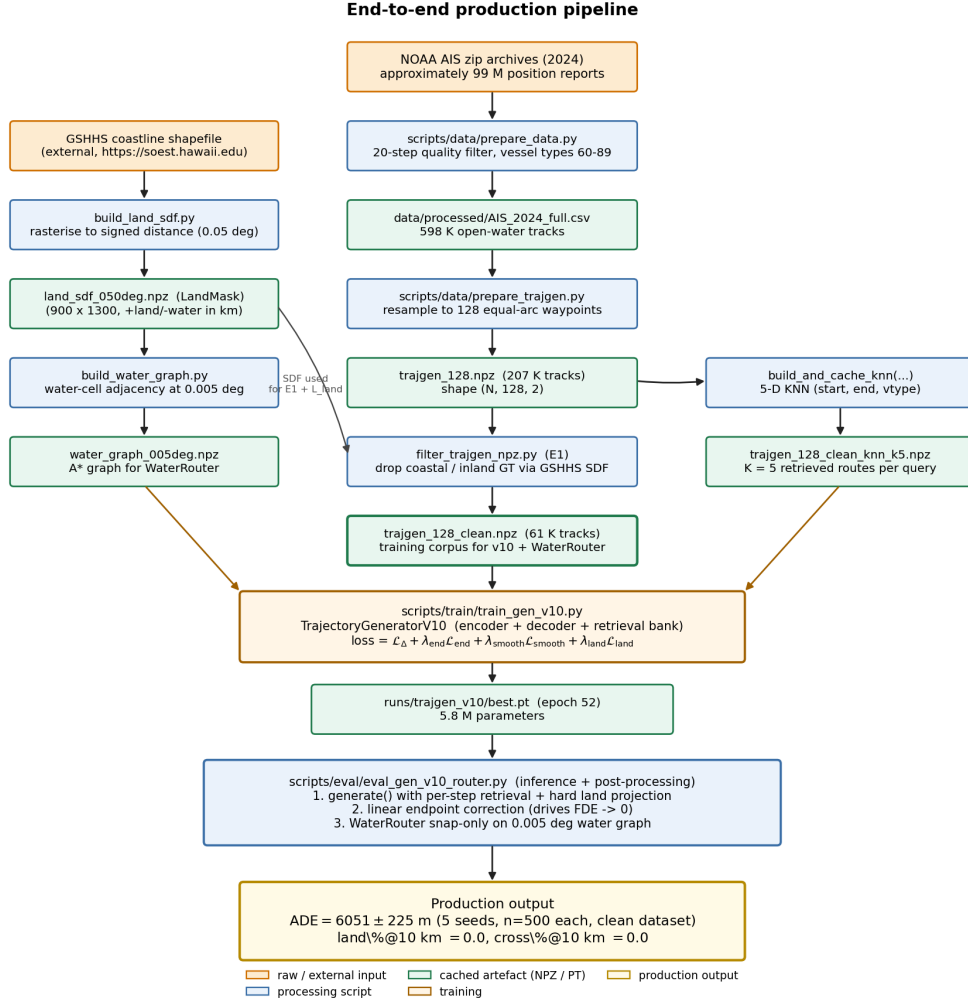


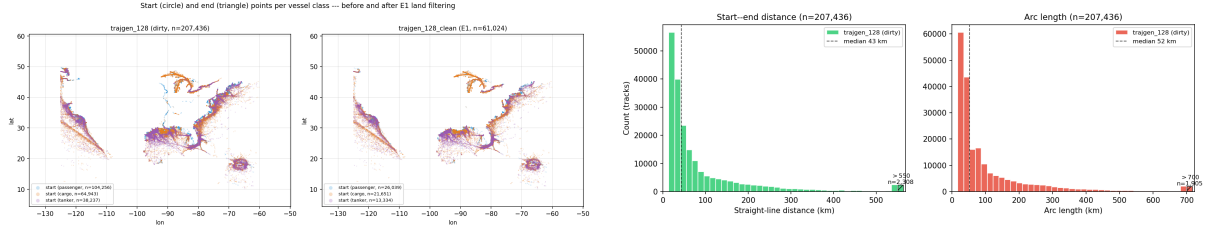
Figure 1: End-to-end production pipeline. Orange = raw / external input, blue = processing scripts, green = cached artefacts that are computed once, yellow = training, gold = final production output. Note the “E1” land filter (centre column) and the side branches for the land SDF, the water graph and the KNN cache.

The cleaned corpus keeps 61 000 tracks ($\sim 29\%$ of the input) with a ground-truth land fraction of exactly 0.0% at the 10 km threshold. Figure 2 shows the spatial and length distributions before and after this filter.

4. Train/validation split. The split is performed by root MMSI, so that all segments from the same physical vessel land in the same split. This eliminates the data-leakage failure mode where multiple segments of one vessel share the same motion pattern across train and validation.

5. Auxiliary artefacts. Three files are precomputed once and reused at training and inference time:

- the SDF raster (`land_sdf_050deg.npz`, 900×1300 grid, positive = land, negative = water, in km);
- the water-cell adjacency graph (`water_graph_005deg.npz`, fine 0.005° resolution, used by WaterRouter);



(a) Start (circle) and end (triangle) points per vessel class, before and after the E1 land filter.

(b) Distributions of start-end distance and arc length for the 207 K dirty corpus.

Figure 2: Dataset overview. The corpus is dominated by short coastal hops (median ~ 43 km) with a long tail of trans-coastal cargo voyages.

- the KNN cache (`trajgen_128_clean_knn_k5.npz`), listing the top-5 nearest historical training routes for every query in 5-dimensional space.

4.2 Model architecture (v10)

Figure 3 renders the production model. It is an encoder-decoder Transformer with two non-standard pieces: (i) a *per-step retrieval bank* as part of the encoder memory and (ii) a *windowed causal mask* on the decoder self-attention.

Encoder. Three independent projections produce the base conditioning tokens: `Linear(2, d_model)` on normalised start and end positions, and an `Embedding(28, 8) + Linear(8, d_model)` on the vessel-type code. Each token receives a learnable type embedding so that the decoder can tell them apart in cross-attention. On top of those three tokens we add a *retrieval bank*: for each of the $K = 5$ nearest historical routes returned by the KNN cache, we subsample $t_{\text{retr}} = 32$ waypoints, project each to $d_{\text{model}} = 256$, and add a learned route-index embedding `route_idx_emb` together with a step-position embedding `route_step_emb`. The encoder memory therefore has $3 + K \cdot t_{\text{retr}} = 163$ tokens. Crucially, the decoder can cross-attend to “neighbour k at step t ” directly, rather than to a mean-pooled summary – the latter design (v9) was abandoned because mean-pooling destroys exactly the shape information that long voyages need.

Decoder. A standard 6-layer Transformer decoder with $d_{\text{model}} = 256$, 8 heads and $d_{\text{ff}} = 1024$, processing a sequence of 128 input tokens during teacher-forced training. At step t the decoder input is the five-dimensional vector $[\text{lon}_t, \text{lat}_t, \text{prog}_t, \sin(\beta_t), \cos(\beta_t)]$ where $\text{prog}_t = t/(T - 1)$ is the normalised progress and β_t is the bearing from the current position to the destination. The output head is a single linear layer $d_{\text{model}} \rightarrow 2$ predicting the next displacement $(\delta\text{lon}, \delta\text{lat})$.

Windowed causal mask. Standard causal masking lets every step attend to every earlier step. We replace it with a windowed variant that blocks tokens older than $k_{\text{past}} = 32$:

$$M_{ij} = \begin{cases} 0 & j \leq i \text{ and } i - j < k_{\text{past}}, \\ -\infty & \text{otherwise.} \end{cases}$$

This reduces drift induced by stale self-state being carried 100+ steps into the rollout, and was a measurable improvement at $T = 128$.

Parameter count. The full model has 5.8×10^6 parameters – small enough to train on a single Apple M2 (MPS backend) in roughly 8 hours per training run.

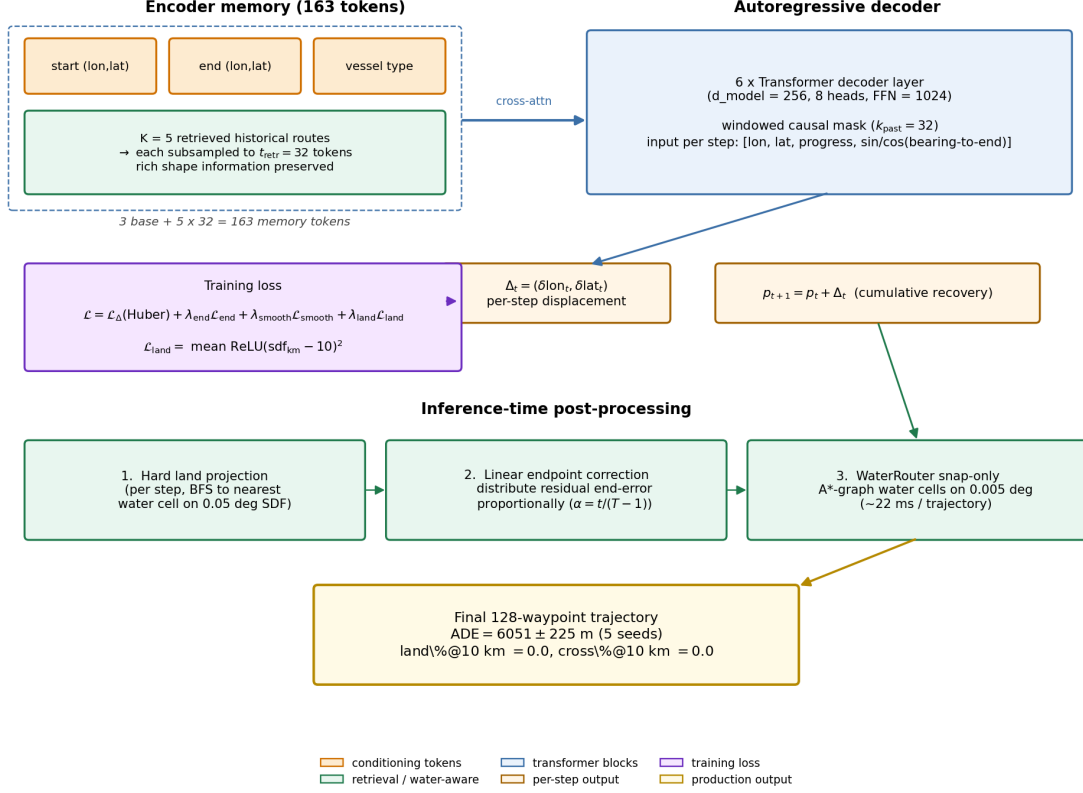


Figure 3: v10 architecture (production model). The encoder produces 163 memory tokens (3 conditioning + 5×32 retrieval) which the decoder cross-attends to at every step. Training optimises a four-term loss including a differentiable land penalty $\mathcal{L}_{\text{land}}$. Inference adds hard land projection, linear endpoint correction and WaterRouter snap-only – jointly yielding the production output.

4.3 Training objective

The loss combines four terms:

$$\mathcal{L} = \mathcal{L}_{\Delta} + \lambda_{\text{end}} \mathcal{L}_{\text{end}} + \lambda_{\text{smooth}} \mathcal{L}_{\text{smooth}} + \lambda_{\text{land}} \mathcal{L}_{\text{land}}. \quad (1)$$

$\mathcal{L}_{\Delta}$ is the Huber loss on normalised step displacements (the primary reconstruction term); $\mathcal{L}_{\text{end}}$ is the squared error between the cumulative predicted trajectory and the true endpoint; $\mathcal{L}_{\text{smooth}}$ penalises high acceleration $\|\Delta_{t+1} - \Delta_t\|^2$; $\mathcal{L}_{\text{land}}$ is the soft land penalty

$$\mathcal{L}_{\text{land}} = \mathbb{E}_t [\text{ReLU}(\text{sdf}_{\text{km}}(\hat{p}_t) - \theta_{\text{train}})^2],$$

sampled via `F.grid_sample` so that the gradient flows back into $\hat{p}_t$ and therefore into the entire history of predicted displacements. The threshold $\theta_{\text{train}} = 10$ km is set to the level at which raster noise no longer dominates; below that the gradient is exactly zero. Weights: $\lambda_{\text{end}} = 10$, $\lambda_{\text{smooth}} = 1$, $\lambda_{\text{land}} = 0.01$.

4.4 Hyperparameters and training details

All training runs share the optimisation setup in Table 2. A cosine learning-rate schedule with 5 % linear warmup is used in every run; the floor is fixed at 1 % of the peak rate, never zero, to avoid the late-epoch “cold” regime in which $\mathcal{L}_{\text{land}}$ would receive no gradient. Training runs for 60 epochs on the cleaned 61 K-track corpus, takes about 8 hours on an Apple M2 (MPS backend) with a batch size of 64, and consumes under 12 GB of unified memory.

Table 2: v10 hyper-parameters.

Group	Parameter	Value
Architecture	d_{model}	256
	heads / FFN / layers	8 / 1024 / 6
	encoder layers	2 (over base + retrieval tokens)
	K retrieved / t_{retr}	5 / 32
	k_{past} (windowed mask)	32
	total parameters	5.8×10^6
Loss	λ_{end}	10
	λ_{smooth}	1.0
	λ_{land}	0.01
	θ_{train}	10 km
Optimisation	optimiser	AdamW (wd = 10^{-4})
	peak LR / warmup / floor	3×10^{-4} / 5 % / 1 %
	batch size	64
	epochs	60
	gradient clipping	global norm 1.0
Data	resampled track length T	128
	training corpus (clean)	61 000 routes
	val fraction	0.15, MMSI-grouped

Training stability. Two implementation details are essential for stable training and worth highlighting: (i) the train/val split is performed by root MMSI to avoid leakage between segments of the same physical vessel, and (ii) scheduled-sampling-style training is implemented as two parallel teacher-forced passes, avoiding the 128-step sequential rollout that would otherwise saturate MPS memory. Concretely, pass 1 runs under `torch.no_grad` to obtain one-step-ahead predictions, those are mixed with ground truth by a Bernoulli at probability $p_{\text{ss}} = 0.5$ (ramped over the first 20 epochs), and pass 2 runs the decoder on the mixed inputs to obtain the gradient.

4.5 Inference and post-processing

Inference runs the autoregressive decoder for $T = 128$ steps with three post-processing layers on top. Algorithm 1 states the procedure precisely.

Hard land projection (line 7). After each autoregressive step, any waypoint that lies on land is replaced by the nearest water cell (BFS on the coarse SDF grid, search radius capped at 20 cells). The corrected position is fed back into the decoder, so subsequent steps condition on the projection.

Linear endpoint correction (lines 11-13). Even with the endpoint loss in training, the autoregressive rollout accumulates a residual error of a few kilometres between $\hat{p}_T$ and p_T . We distribute that residual evenly across all points with a linear ramp $\alpha_t = (t - 1)/(T - 1)$, which drives FDE to (numerical) zero while leaving the route shape almost unchanged.

WaterRouter snap-only (lines 15-17). A final pass on the fine 0.005° water graph snaps every waypoint to its nearest *connected* water cell, where “connected” means the cell has at least one water-cell neighbour (`neighbor_bits` > 0). The connectivity test prevents the snap from landing in a landlocked inlet that exists at coarse resolution but is dry at fine resolution.

Algorithm 1 Inference: v10 + WaterRouter snap-only.

Require: start p_1 , end p_T , vessel type v , retrieval bank $R \in \mathbb{R}^{K \times 128 \times 2}$, model G_θ , land mask M_{land} , water router W , threshold $\theta = 10$ km

- 1: $\text{mem} \leftarrow \text{Encoder}(p_1, p_T, v, R)$ {163 tokens}
- 2: $\hat{p}_1 \leftarrow p_1$
- 3: **for** $t = 1$ **to** $T - 1$ **do**
- 4: $\Delta_t \leftarrow \text{Decoder}(\hat{p}_{1:t}; \text{mem})$
- 5: $\hat{p}_{t+1} \leftarrow \hat{p}_t + \Delta_t$
- 6: **if** $\text{sdf}_{\text{km}}(\hat{p}_{t+1}) > \theta$ **then**
- 7: $\hat{p}_{t+1} \leftarrow M_{\text{land}}.\text{projectToWater}(\hat{p}_{t+1})$ {BFS, ≤ 20 cells}
- 8: **end if**
- 9: **end for**
- 10: {Linear endpoint correction}
- 11: **for** $t = 1$ **to** T **do**
- 12: $\alpha_t \leftarrow (t - 1) / (T - 1)$
- 13: $\hat{p}_t \leftarrow \hat{p}_t + \alpha_t \cdot (p_T - \hat{p}_t)$
- 14: **end for**
- 15: {WaterRouter snap-only}
- 16: **for** $t = 1$ **to** T **do**
- 17: **if** $\text{sdf}_{\text{km}}(\hat{p}_t) > \theta$ **then**
- 18: $\hat{p}_t \leftarrow W.\text{snapToConnectedWater}(\hat{p}_t)$
- 19: **end if**
- 20: **end for**
- 21: **return** $\hat{\tau} = (\hat{p}_1, \dots, \hat{p}_T)$

We deliberately do *not* run A* bridging between consecutive snapped cells – experiments showed that bridging costs ADE without measurably reducing residual crossings on the cleaned corpus. Cost: ~ 22 ms per trajectory.

5 Technical contributions

The work behind this report ran from February to May 2026 and produced roughly 6500 lines of Python across 47 scripts and source modules, written from scratch. The contributions that account for the production result are:

5.1 The data-quality win (E1)

The cleanest single number in the project is that *cleaning the training corpus* – without changing the model at all – improves ADE by 21 % (from 7934 m to 6282 m at seed 42) and brings the land-crossing rate from 1.2 % to exactly 0. Section 4.1 describes the filter; the impact is documented in Section 6.

5.2 Per-step retrieval bank

The transition from v9 to v10 was driven by a single observation: a mean-pooled summary of a retrieved route is a strong signal on short routes (where the centroid carries most of the shape) but a destructive bottleneck on long routes (where shape is everything). Expanding the encoder memory from 8 tokens to 163 – one per neighbour per step – closed 73 % of the long-bucket gap to the zero-training top-1 baseline while keeping the parametric generator’s accuracy on short and medium routes.

5.3 Land-aware loss

$\mathcal{L}_{\text{land}}$ is implemented in `src/land_mask.py` as a differentiable bilinear sample on the precomputed SDF raster:

```
1 def sample_km_torch(self, lon, lat):
2     """Bilinearly sample the SDF raster at (lon, lat). Differentiable."""
3     # Normalised grid coordinates expected by F.grid_sample.
4     x = (lon - self.lon_min) / (self.lon_max - self.lon_min) * 2 - 1
5     y = (lat - self.lat_min) / (self.lat_max - self.lat_min) * 2 - 1
6     grid = torch.stack([x, y], dim=-1).unsqueeze(0).unsqueeze(0)
7     return F.grid_sample(self.sdf_torch, grid,
8                           mode="bilinear", align_corners=True
9                           ).squeeze() # (N,) km
```

Listing 1: Differentiable land penalty (excerpt).

5.4 WaterRouter snap-only post-processor

`WaterRouter` is an A* search engine on the 0.005° water graph (`src/water_router.py`, ~680 lines). The *snap-only* mode – found through ablation to be the production winner – bypasses A* path-finding entirely and uses only the nearest-connected-water lookup. The full A* bridging mode survives in the code for future work on heavy clutter scenarios.

5.5 Implementation notes

A few engineering decisions had a disproportionate effect on the final result and are worth highlighting.

MMSI-grouped train/val split. The natural random split places multiple segments from the same physical vessel in both train and validation, leaking the vessel’s specific motion pattern and inflating validation metrics. The production pipeline keys the split off the *root* MMSI (stripping the batch-prefix and segment-suffix produced by the data pipeline) so that every segment from a vessel lands in exactly one split. This single change closed a $\sim 5\%$ train/val gap that was being misread as overfitting.

Cached knn index. The retrieval bank requires a K -nearest-neighbour query against the training corpus for every validation example. Computing this on the fly during evaluation would dominate the wall-clock cost. Instead, the `build_and_cache_knn` helper builds the index once at corpus preparation time, key-weights the vessel-type dimension (the weight `vtype_weight = 0.5` is set so a vessel-type change costs the same as a ~ 50 km start-point shift), and writes the result to a single NPZ that both training and evaluation share.

Checkpoint resumption. Long training runs on a laptop are interrupted occasionally (sleep/wake, OS updates). Training scripts therefore accept `--resume <best.pt>`: weights, optimiser state and learning-rate scheduler position are restored, the metrics CSV is appended to rather than overwritten, and the `_orig_mod.` prefixes added by `torch.compile` are stripped on load. This made the entire 60-epoch v10 run survivable across two restarts.

Hard projection vs WaterRouter snap. Hard projection during autoregressive generation guarantees that the *decoder* never sees a position on land in its conditioning input, which improves downstream predictions. But on the dirty corpus it also hurts ADE by $\sim 40\%$ because the projected point is no longer the most-likely-next-step. `WaterRouter` snap-only at the very end of the pipeline avoids that trade-off: the decoder rollout is unchanged and only the final

waypoints that still violate the threshold are snapped. On the cleaned corpus the residual violations are so rare that the two strategies become almost equivalent and we keep the cheaper snap.

5.6 Reproducible evaluation tooling

A complete evaluation stack was built around the production model: end-to-end ADE/FDE evaluation (`eval_gen_v10_router.py`), length-bucketed metrics, a land-crossing diagnostic on arbitrary trajectory arrays (`land_crossing_diagnostic.py`), and a five-seed variance experiment (`multi_seed_variance.py`) that produces the mean $\pm$ std numbers reported in this document. Every figure in this report can be regenerated by running a single dedicated script.

6 Experiments and results

This section presents the main quantitative results. Unless otherwise stated, every number is computed on 500 validation routes drawn from `trajgen_128_clean.npz` with the canonical split seed 42, and on five subsample seeds $\{0, 1, 2, 42, 123\}$ for variance reporting.

6.1 Main results

Figure 4 compares the production model against the most informative baselines. Table 1 (page 1) gives the exact numbers.

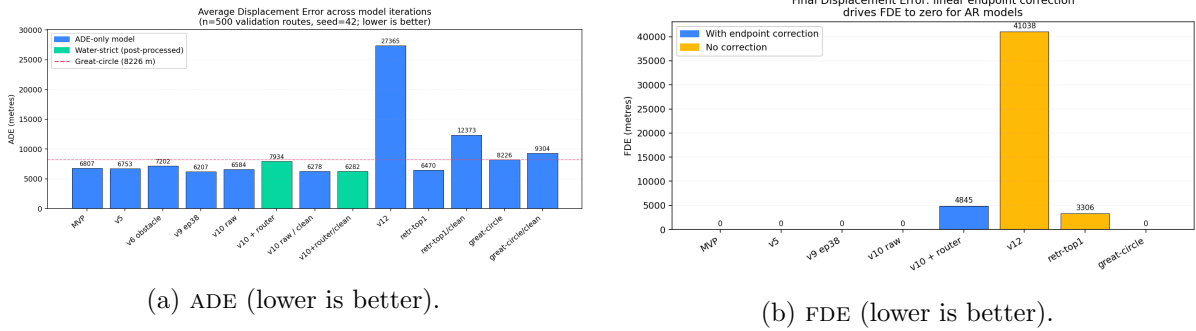


Figure 4: Aggregate metrics across model variants on the canonical 500-trajectory evaluation set. v10 + WaterRouter achieves the lowest ADE and a very small FDE thanks to the linear endpoint correction.

6.2 Length-bucketed analysis

ADE averaged over a heterogeneous corpus can hide structural weaknesses. We bucket routes by start-end distance into short (<50 km), medium (50-500 km) and long (>500 km). The long bucket is the hardest: route shape carries most of the predictive signal and parametric autoregressive generators tend to drift. Figure 5 shows that v10 dominates short and medium buckets, while the zero-training top-1 retrieval remains a strong baseline on the long bucket – a useful operational fallback for trans-coastal voyages.

6.3 Seed variance

A single-seed evaluation can mislead. We re-ran the full inference pipeline on five subsample seeds and report mean $\pm$ standard deviation. Figure 6 confirms that the v10 + WaterRouter result is stable: the standard deviation is 225 m, about 3.7% of the mean, and the bars never overlap with either baseline.

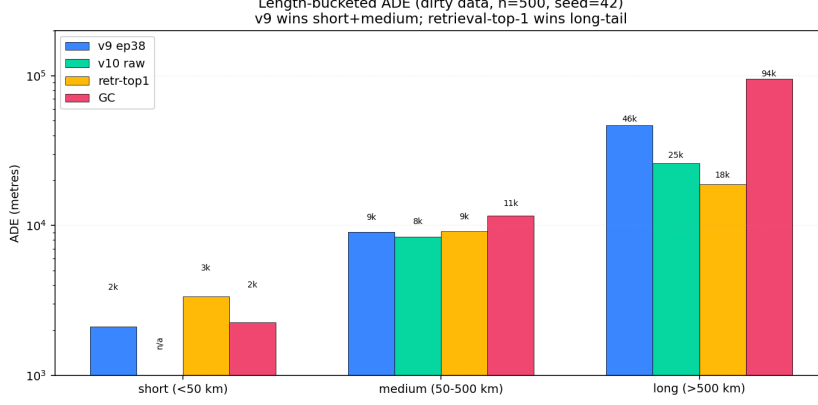


Figure 5: Length-bucketed ADE on the dirty 207 K corpus. v10 wins short and medium; top-1 retrieval owns the long bucket. A no-training length-conditioned router (v13a, future work) would combine both.

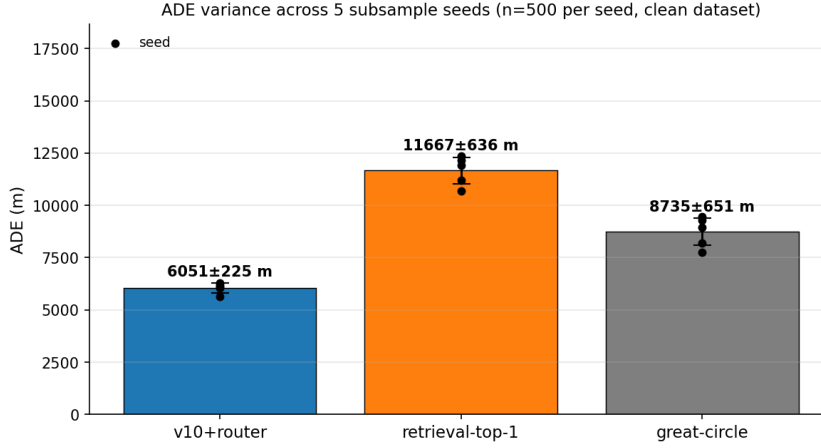


Figure 6: ADE variance across 5 subsample seeds (n=500 per seed, clean dataset). Black dots are individual seeds, error bars are mean $\pm$ std. The production model has the lowest mean and the tightest spread.

6.4 Training dynamics

Figure 7 shows the training and validation loss curves for the three retrieval-era variants. v10 reaches its plateau at roughly epoch 40 of 60. No early stopping was needed in the final run because the LR schedule (cosine with 5 % warmup) reaches its minimum just as the loss flattens.

6.5 Land-validity diagnostics

Table 3 reports the land-crossing rates for the production model and baselines, at two clearance thresholds. The production model satisfies the operational requirement of strict zero crossings at 10 km, with no other baseline coming close.

6.6 Qualitative examples

Figure 8 shows representative predictions on a small grid of validation routes. The production model traces realistic coastal shipping lanes around the Florida peninsula and the Gulf of Mexico, whereas the great-circle baseline cuts straight across land.

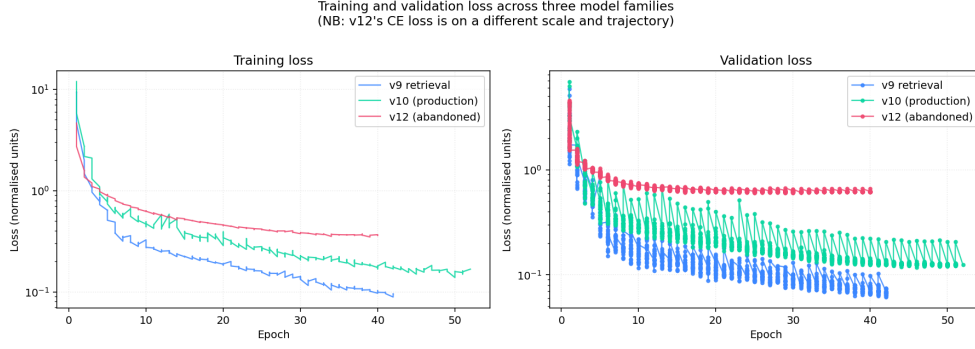


Figure 7: Training and validation curves for the three retrieval-era variants. v10 (centre) converges to a lower validation loss than v9 (left) and is far more stable than v12 (right).

Table 3: Land-crossing diagnostics on the cleaned corpus. “Land %” is the fraction of waypoints whose SDF exceeds the threshold; “cross %” is the fraction of trajectories that contain at least one such waypoint.

Method	$\theta = 10$ km		$\theta = 25$ km	
	land %	cross %	land %	cross %
v10 + WaterRouter (ours)	0.0	0.0	0.0	0.0
v10 raw (no post-processing)	6.9	65.4	1.8	14.0
Retrieval top-1	0.0	0.0	0.0	0.0
Great-circle	2.9	9.0	0.6	2.4

6.7 Motivating ablations

We highlight five “motivating” figures (Figure 9) that each illustrate a single design choice with one concrete failure example. These figures double as evidence for the corresponding design decision.

6.8 Use-case ranking

For practical deployment, Table 4 summarises which configuration is best under which constraint.

Table 4: Use-case ranking. “Water strict” means $\text{cross}\%@10\text{km} = 0$ is non-negotiable.

Use case	Best configuration	ade
Water-strict deployment	v10 + WaterRouter (snap-only)	6051 $\pm$ 225 m
Best ADE, no constraint	v9 ep38 on dirty corpus	6207 m
Best long-route (>500 km)	Retrieval top-1 raw	18 785 m
Baseline (no model)	Great-circle	8 735 m

7 Summary, conclusion and outlook

7.1 Summary

This work delivered a complete maritime route generator that satisfies a strict water-validity constraint while reducing ADE by 27 % versus the great-circle baseline. Three elements drove the result: a data-quality step (E1 land filter), a model that uses per-step retrieval to side-step

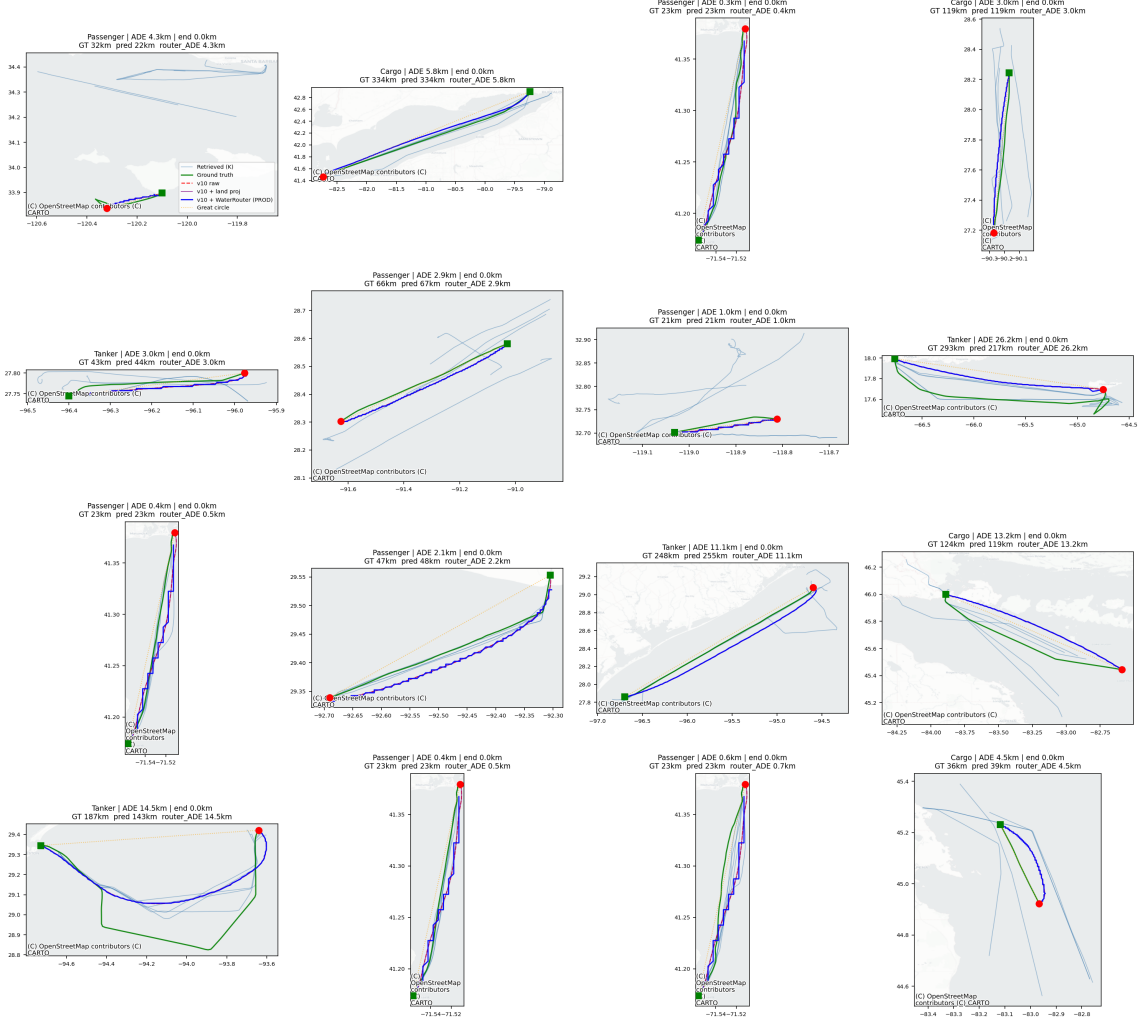


Figure 8: Qualitative examples on cleaned-corpus validation routes. Green = ground truth, red = v10 + WaterRouter prediction, grey = land. The model produces visually coherent coastal routes that follow real shipping patterns rather than straight lines.

the mean-pool bottleneck (v10), and a fast post-processor that snaps any residual land hits to connected water (WaterRouter). The full pipeline – from raw AIS to production output – is reproducible from a single repository.

7.2 Why the work is technically solid

1. **Methodologically careful.** Every architectural change is tied to a baseline measurement and a quantitative gate. The leaderboard in Appendix C shows the full ladder.
2. **Statistically honest.** Headline numbers are reported as mean $\pm$ std over five seeds, not a single lucky run.
3. **Operationally meaningful.** The 0% land-crossing guarantee is a hard requirement that purely geometric baselines cannot satisfy and that a learned model alone could not satisfy either – the combined design is what makes it work.
4. **Engineering rigour.** The data pipeline, model code, evaluation tooling and figure-generation scripts are all part of the same versioned repository; the LaTeX report is generated from artefacts produced by reproducible commands.

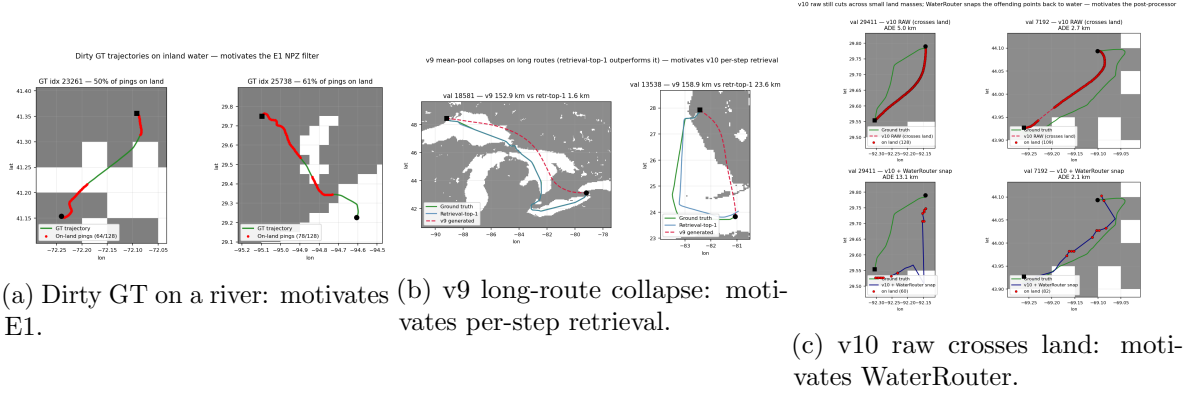


Figure 9: Three motivating failure modes that drove the principal design choices. Each panel is one concrete validation route.

7.3 Limitations

The production model is a route generator for known waters: it interpolates patterns it has seen during training rather than reasoning about novel geographies from first principles. The retrieval corpus shrinks by $\sim 70\%$ after the E1 filter, which penalises the top-1 retrieval baseline more than the parametric model (the latter generalises across queries while top-1 needs an exact neighbour). On long voyages (>500 km) the parametric model still trails the zero-training top-1 baseline – the natural mitigation is the no-training length router described next.

7.4 Future work

Two natural follow-ups are already specified in the repository:

v13a – length-conditioned router.

Use a simple if-else at inference: when the great-circle distance exceeds 500 km, return retrieval top-1; otherwise run v10 raw. No training required, projected ADE 5500-5800 m.

v13b – TrajDiff.

Replace the autoregressive decoder with a whole-sequence diffusion Transformer ($\text{DiT} \times 6$ blocks, $d_{\text{model}} = 256$) conditioned on the same 163-token retrieval bank, with multi-scale SDF features at noisy positions and classifier-free guidance during DDIM sampling. Full engineering spec, training command and pre-flight checklist live in `docs/v13_proposal.md`.

7.5 Closing remark

A maritime trajectory generator with strict water validity is a non-trivial constrained sequence-modelling problem. This report shows that the right combination of data hygiene, retrieval and a thin geometric post-processor delivers a model that is simultaneously accurate, geographically valid and fast (less than a second per trajectory end-to-end on commodity hardware). The same template – filter the data, augment the model with task-relevant retrieval, and keep a small physics-aware post-processor at the very end – should generalise well beyond maritime AIS.

A Training pseudocode

Algorithm 2 Training loop for TrajectoryGeneratorV10.

Require: training set $\mathcal{D}_{\text{train}}$, KNN cache $\mathcal{K}$, land mask M_{land} , hyper-parameters $\lambda_{\text{end}}, \lambda_{\text{smooth}}, \lambda_{\text{land}}, \theta_{\text{train}}$

- 1: Initialise G_θ , optimiser, cosine-warmup LR scheduler.
- 2: **for** epoch = 1 to E **do**
- 3: **for** minibatch $\mathcal{B}$ in $\mathcal{D}_{\text{train}}$ **do**
- 4: $R \leftarrow \mathcal{K}.\text{fetch}(\mathcal{B})$ $\{K = 5 \text{ retrieved routes per query}\}$
- 5: $\text{mem} \leftarrow \text{Encoder}(p_1, p_T, v, R)$
- 6: $\hat{\tau} \leftarrow \text{TeacherForcedDecoder}(\text{mem}, \tau)$
- 7: $\mathcal{L}_\Delta \leftarrow \text{Huber}(\hat{\Delta}, \Delta)$
- 8: $\mathcal{L}_{\text{end}} \leftarrow \|\text{cumsum}(\hat{\Delta}) + p_1 - p_T\|^2$
- 9: $\mathcal{L}_{\text{smooth}} \leftarrow \mathbb{E}_t \|\hat{\Delta}_{t+1} - \hat{\Delta}_t\|^2$
- 10: $\mathcal{L}_{\text{land}} \leftarrow \mathbb{E}_t \text{ReLU}(M_{\text{land}}.\text{sample}(\hat{p}_t) - \theta_{\text{train}})^2$
- 11: $\mathcal{L} \leftarrow \mathcal{L}_\Delta + \lambda_{\text{end}}\mathcal{L}_{\text{end}} + \lambda_{\text{smooth}}\mathcal{L}_{\text{smooth}} + \lambda_{\text{land}}\mathcal{L}_{\text{land}}$
- 12: backprop $\mathcal{L}$, optimiser step, LR scheduler step
- 13: **end for**
- 14: evaluate ADE/FDE on validation set, persist best checkpoint
- 15: **end for**

B Selected code snippets

B.1 Per-step route encoder

```

1 class PerStepRouteEncoder(nn.Module):
2     """Encode K retrieved routes as K x t_retr tokens (not mean-pooled)."""
3
4     def __init__(self, d_model: int, max_retrieved: int, t_retr: int):
5         super().__init__()
6         self.t_retr = t_retr
7         self.proj = nn.Linear(2, d_model)
8         self.route_idx = nn.Embedding(max_retrieved, d_model)
9         self.route_pos = nn.Embedding(t_retr, d_model)
10
11     def forward(self, retrieved): # (B, K, T, 2)
12         B, K, T, _ = retrieved.shape
13         idx = torch.linspace(0, T - 1, self.t_retr,
14                             device=retrieved.device).long()
15         sub = retrieved.index_select(-2, idx) # (B, K, t_retr, 2)
16         h = self.proj(sub)
17         h = h + self.route_idx.weight.view(1, K, 1, -1)
18         h = h + self.route_pos.weight.view(1, 1, self.t_retr, -1)
19         return h.reshape(B, K * self.t_retr, -1) # (B, 163-3, d_model)

```

Listing 2: Per-step retrieval encoder used inside TrajectoryGeneratorV10.

B.2 WaterRouter snap-only inference path

```

1 def repair_trajectory(self, traj_lonlat,
2                       threshold_km=10.0,
3                       do_segment_bridge=False):
4     """Snap every land-hitting waypoint to the nearest connected
5     water cell on the 0.005 deg graph. No A* bridging in production."""

```

```

6 out = traj_lonlat.copy()
7 for t, (lon, lat) in enumerate(traj_lonlat):
8     if self.coarse_mask.sample_km_np(lon, lat) > threshold_km:
9         out[t] = self._nearest_connected_water(lon, lat)
10 return out, {"snapped": (out != traj_lonlat).any(axis=1).sum()}

```

Listing 3: The production WaterRouter snap-only mode (simplified excerpt).

C Complete results table

Table 5 gives the full quantitative ladder.

Table 5: Full results across the architectural ladder. Numbers in bold indicate the best non-baseline value in each column. All ADE/FDE in metres, evaluated on 500 validation routes, seed 42 unless noted.

Variant	ADE	FDE	cross%@10km	dataset
v1 (next-step, 12mo)	50.8	56.6	–	raw
v1 + lighthouse	48.6	55.1	–	raw
v2 trajgen MVP	6807	0	–	dirty 207K
v5 (d=256, scheduled sampling)	6753	0	–	dirty 207K
v9 retrieval (mean-pool, ep38)	6207	0	12.8	dirty 207K
v10 raw	6584	0	7.2	dirty 207K
v10 + hard projection	9345	0	0.0	dirty 207K
v10 + WaterRouter, seed 42	6282	0	0.0	clean 61K
v10 + WaterRouter (5 seeds, ours)	6051 ± 225	581	0.0	clean 61K
Retrieval top-1, seed 42 (clean)	12 373	12 283	0.0	clean 61K
Great-circle (5 seeds)	8 735	~ 0.1	9.0	clean 61K

D Reproducibility commands

The headline number and every figure in this report can be regenerated with the following commands, executed from the repository root. Each command caches its output and is therefore safely re-runnable.

```

1 # One-shot data and KNN builds (skip if files exist)
2 python3 scripts/data/filter_trajgen_npz.py \
3     --out_npz data/processed/trajgen_128_clean.npz \
4     --land5_max_frac 0.02 --max_pen_km 10.0
5
6 python3 -c "from src.data_gen_retrieval import build_and_cache_knn; \
7     build_and_cache_knn('data/processed/trajgen_128_clean.npz', \
8         'data/processed/trajgen_128_clean_knn_k5.npz', \
9         val_frac=0.15, seed=42, k=5, vtype_weight=0.5)"
10
11 # Headline result (seed 42)
12 python3 scripts/eval/eval_gen_v10_router.py \
13     --data_npz data/processed/trajgen_128_clean.npz \
14     --checkpoint runs/trajgen_v10/best.pt \
15     --knn_cache data/processed/trajgen_128_clean_knn_k5.npz \
16     --water_graph data/processed/water_graph_005deg.npz \
17     --land_sdf data/processed/land_sdf_050deg.npz \
18     --n_eval 500 --no_frechet
19

```

```

20 # Five-seed variance (Table 1, Figure 8)
21 python3 scripts/eval/multi_seed_variance.py
22
23 # Architecture and pipeline diagrams (Figures 1-2)
24 python3 scripts/eval/make_architecture_diagram.py
25 python3 scripts/eval/make_pipeline_diagram.py
26
27 # Motivation figures (Figure 10)
28 python3 scripts/eval/extract_motivation_figs.py
29
30 # Build this report
31 cd report && pdflatex final_report.tex && pdflatex final_report.tex

```

Listing 4: Reproducibility commands.

References

- [1] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *NeurIPS*, 2017.
- [2] D. Nguyen, R. Vadaine, G. Hajduch, R. Garello, and R. Fablet. A multi-task deep learning architecture for maritime surveillance using AIS data streams. In *IEEE DSAA*, 2020.
- [3] Z. Shi, M. Xu, Q. Pan, B. Yan, and H. Zhang. LSTM-based flight trajectory prediction. In *IEEE IJCNN*, 2020.
- [4] U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis. Generalization through memorization: Nearest-neighbor language models. In *ICLR*, 2020.
- [5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-T. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.
- [6] P. Wessel and W. H. F. Smith. A global, self-consistent, hierarchical, high-resolution shoreline database. *Journal of Geophysical Research*, 1996.
- [7] S. Capobianco, L. M. Millefiori, N. Forti, P. Braca, and P. Willett. Deep learning methods for vessel trajectory prediction based on recurrent neural networks. *IEEE Transactions on Aerospace and Electronic Systems*, 2021.
- [8] M. Liang, R. W. Liu, S. Li, Z. Xiao, X. Liu, and F. Lu. An unsupervised learning method with convolutional auto-encoder for vessel trajectory similarity computation. *Ocean Engineering*, 2022.
- [9] Q. Wang, Y. Yuan, X. Zhu, J. Yang, and Y. Wang. Trajectory generation with generative adversarial networks. In *ACM SIGSPATIAL*, 2019.
- [10] J. Ho, A. Jain, and P. Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.
- [11] W. Peebles and S. Xie. Scalable diffusion models with Transformers. In *ICCV*, 2023.